

实验 5 路径规划

一、实验目的

- 1、理解自主导航中路径规划算法原理，掌握全局路径规划（如 A*、Dijkstra 算法）与局部路径规划（如 DWA、TEB 算法）的分层架构与协同工作机制。
- 2、深入理解代价地图在导航系统中的核心作用，掌握其如何通过多层数据融合（如静态地图层、实时障碍物层、膨胀层等）将物理环境转化为通行代价值。
- 3、熟悉 ROS 导航栈的整体架构，重点掌握核心控制节点 move_base 的运行逻辑及其与定位系统（如 AMCL）、传感器驱动、底盘控制器之间的数据交互关系。
- 4、了解 ROS 中实现长时任务交互的 Action 通信机制，理解 Action（动作）通信与 Topic（话题）、Service（服务）通信的本质区别。

二、实验环境与器材

- 1、实验硬件设备
计算机、移动机器人导航平台
- 2、实验软件配置
Ubuntu20.04, ros-noetic
- 3、实验场景
教学实验场地

三、实验原理

1. 路径规划算法

在移动机器人自主导航系统中，路径规划是实现机器人从起点安全、高效到达目标点的核心技术。路径规划在自主导航系统中扮演着“大脑”的角色，它不仅要解决“怎么走”的问题，还要解决“如何避障”和“如何适应动态环境”的问题。一个鲁棒的导航系统通常采用分层规划架构，将复杂的全局寻路任务分解为宏观路线制定（全局路径规划）与微观实时控制（局部路径规划）两个层面，从而在保证路径最优性的同时，兼顾了系统的实时性与安全性。

1) 全局路径规划算法

全局路径规划器（Global Planner）的主要任务是在已知环境信息的前提下（即基于预先构建好的静态地图，如实验 3 中 SLAM 生成的栅格地图），计算出一条从当前起始点到最终目标点的最优宏观路径。ROS 导航栈中最常用的全局规划算法包括 Dijkstra 算法和 A* 算法。

Dijkstra 算法：用于查找图中某个顶点到其它所有顶点的最短路径，该算法既适用于无向加权图，也适用于有向加权图。

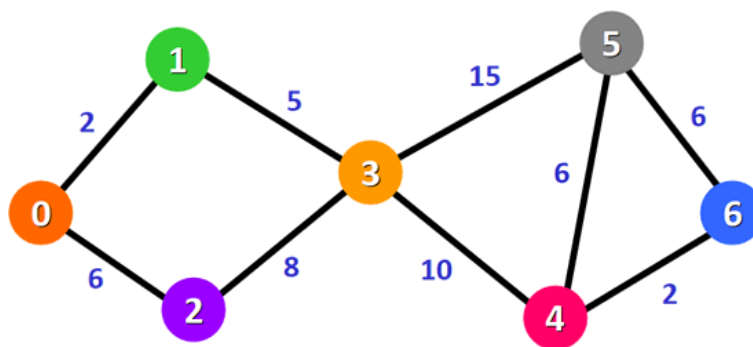


图 1 无向加权图

表 1 最短路径

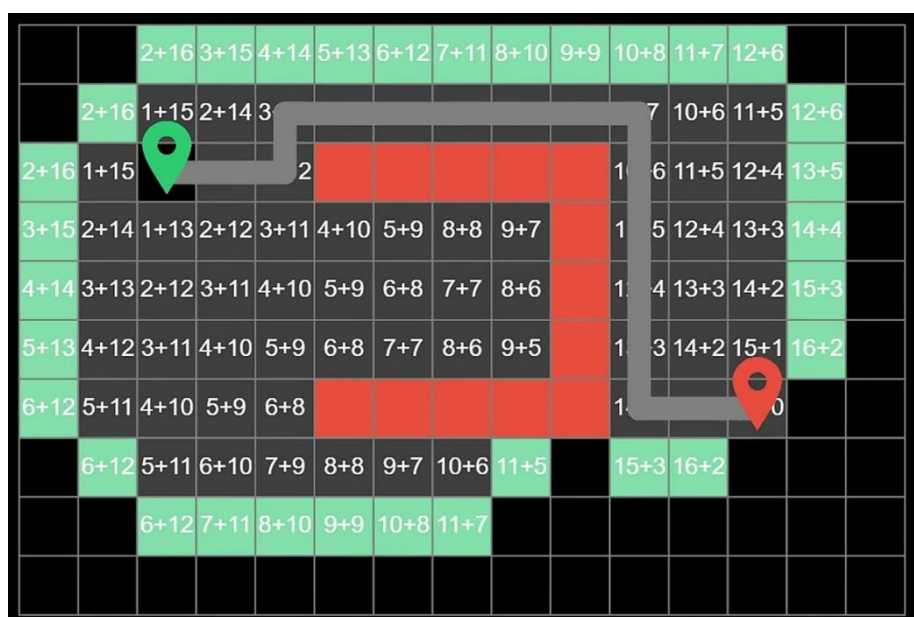
	1	2	3	4	5	6
总权值	2	6	7	17	22	19
路径	0-1	0-2	0-1-3	0-1-3-4	0-1-3-5	0-1-3-4-6

A*算法：在 Dijkstra 算法的基础上加入了启发式函数（评估当前点到终点的距离），搜索速度更快，更适合大范围的地图规划，虽然理论上不一定是最短路径，但在实际机器人导航中效果极佳。

A*算法通过下面这个函数计算每个节点的优先级：

$$f(n) = g(n) + h(n)$$

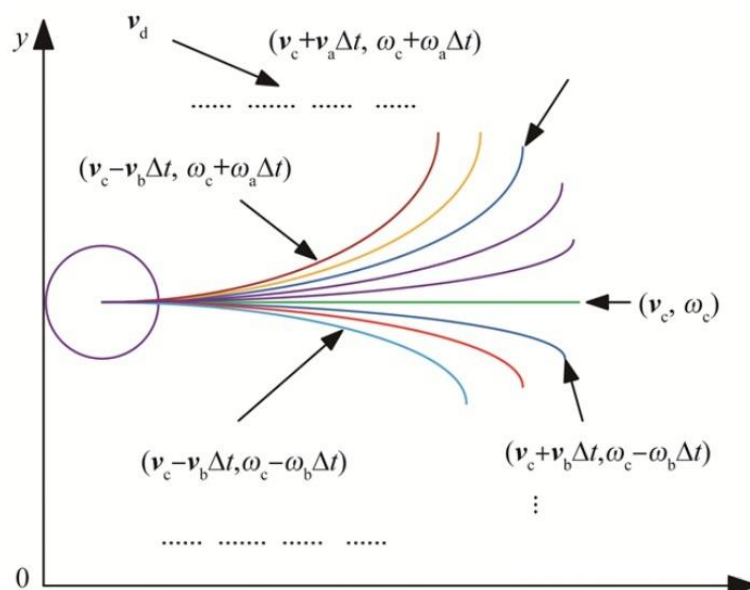
其中， $g(n)$ 是起点到当前节点 n 的距离，即实际代价； $h(n)$ 是当前节点 n 到终点的预估代价，也是 A*算法的启发函数； $f(n)$ 是节点 n 的综合优先级，选择下一个要遍历的节点时，会选取综合优先级最高（或权值最小）的节点。



2) 局部路径规划算法

局部路径规划器（Local Planner）的核心任务是在全局路径的引导下，结合机器人实时的传感器数据（如激光雷达、深度相机），计算机器人在下一时刻的具体运动指令（线速度和角速度）。

动态窗口法（Dynamic Window Approach, DWA）是 ROS 导航栈中最经典、应用最广泛的局部路径规划（避障）算法，核心思想是在机器人的速度空间（线速度 v 和角速度 w ）中，采样出多个可行的速度组合，并根据这些组合模拟出机器人在未来短时间内会走出的不同轨迹，然后根据评价函数（如距离障碍物的远近、朝向目标的程度、消耗时间等）从中选出最优速度指令去执行。



3) 全局与局部规划器的协作机制

全局规划器与局部规划器并非孤立工作，而是相互协作共同维持机器人的导航任务。全局规划器输出的路径是局部规划器的“参考输入”。局部规划器在计算速度指令时，会尽量贴合全局路径前进。可以将这种关系比作车载 GPS 导航（全局）与驾驶员（局部）：GPS 规划出走哪条路，而驾驶员负责控制方向盘和油门，避开路上的行人和车辆，同时尽量不偏离 GPS 规划的路线。

此外，两者之间还存在一个闭环反馈机制：

正常跟随：在无障碍物阻挡时，局部规划器指挥机器人沿着全局路径行驶。

局部避障：当遇到临时障碍物时，局部规划器自主规划绕行轨迹，此时机器人可能会暂时偏离全局路径。

全局重规划：如果障碍物完全堵死了全局路径，或者机器人偏离路径过远导致局部规划器无法找到回归路线，局部规划器会向上层反馈失败信号。此时，全局规划器会被重新触发，基于当前机器人位置重新计算一条绕过阻塞区域的全新路径。

ROS 导航栈通过全局路径规划与局部路径规划的有机结合，完美解决了移动机器人在复杂环境下的导航难题。全局规划器利用静态地图保证了路径的宏观最优性，确立了导航的大方向；局部规划器利用实时传感器数据保证了运动的局部安全性，解决了动态避障问题。两者相辅相成，缺一不可，共同构成了机器人自主移动的智能基石。

2. 代价地图

从上述内容中可以看出，在机器人导航系统中路径规划算法的实现依赖地图，虽然地图的构建在实验 2 的 SLAM 算法中已经完成，但它是是不可以直接在路径规划器中使用的。因为 SLAM 构建的地图只是用灰度值标记障碍物存在的概率，无障碍物区域并不一定是安全地带，如在靠近障碍物的边缘处，虽然是空闲区域，但机器人在这些区域移动时极可能会与障碍物产生剐蹭或碰撞。此外，SLAM 构建的地图是静态地图，而导航过程中，障碍物信息是可变的，原有的障碍物可能被移走，也可能出现新的障碍物，因此移动机器人在自主导航中需要时时的获取障碍物信息。

针对上述情况，导航实现过程中需要在静态地图的基础上添加一些辅助信息，比如时时获取的障碍物数据，基于障碍物生成的膨胀区等，从而辅助路径规划算法避开障碍物，实现安全导航。

1) 代价地图的层级结构

ROS 中的代价地图采用层级结构叠加构建，一般由以下几层构成：

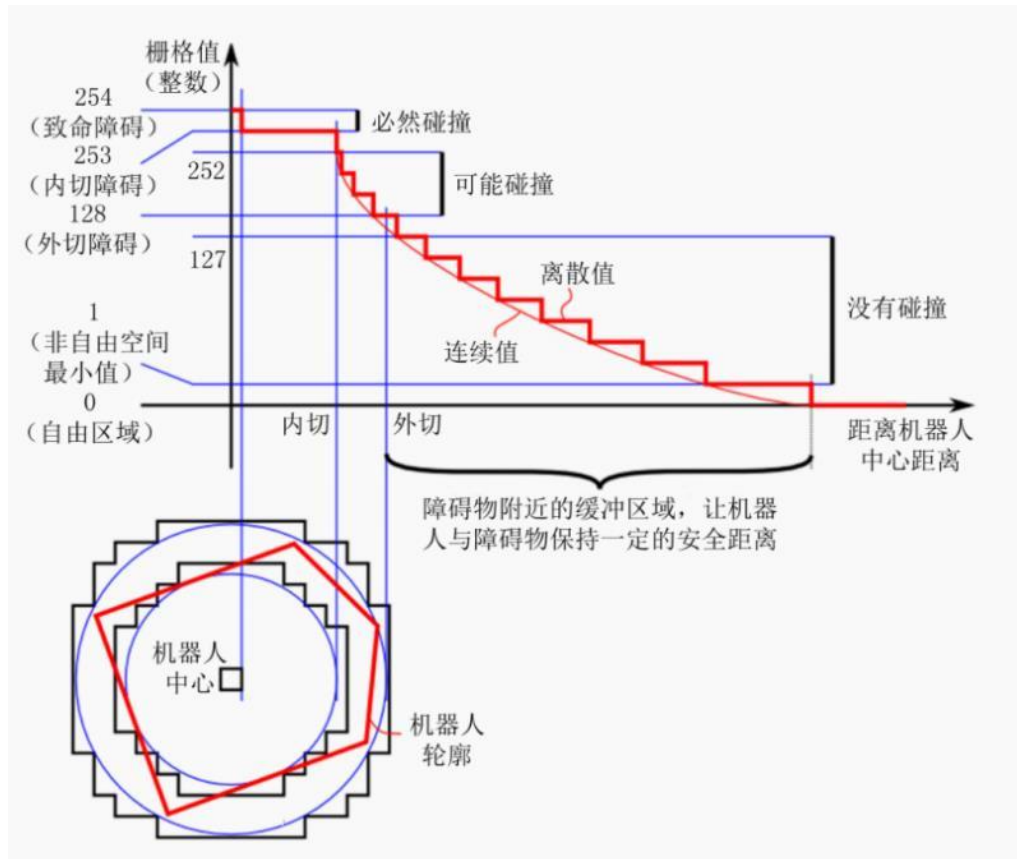
- 静态地图层 (Static Layer)：加载预先构建的静态地图（如 SLAM 生成的地图），标识固定障碍物。
- 障碍物层 (Obstacle Layer)：实时融合激光雷达、深度相机等传感器数据，动态标记移动或新出现的障碍物。
- 膨胀层 (Inflation Layer)：在以上两层地图上标记的障碍物周围生成膨胀区域，形成安全缓冲区，以避免机器人的外壳与障碍物发生碰撞。
- 其他可选层：如体素层 (Voxel Layer，用于 3D 数据处理)，或自定义层。

导航系统由全局和局部两层路径规划组成，并相应地构建了全局与局部两张代价地图：

- 全局代价地图 (Global Costmap)：用于全局路径规划，更新频率较低，覆盖范围大。
- 局部代价地图 (Local Costmap)：用于局部避障和轨迹优化，更新频率高，聚焦机器人周围小范围区域。

2) 代价值含义

代价地图与静态栅格地图一样，也是以二维栅格形式组成的数据，每个栅格的代价值通常为 0 到 255 之间的整数，具体含义如下图所示，横轴是距离机器人中心的距离，纵轴是代价地图中栅格的灰度值：



致命障碍：栅格值 254，此时障碍物与机器人中心重叠，必然发生碰撞。

内切障碍：栅格值 253，此时障碍物处于机器人的内切圆内，必然发生碰撞；

外切障碍：栅格值为 $[128, 252]$ ，此时障碍物处于其机器人的外切圆内，处于碰撞临界，不一定发生碰撞；

非自由空间：栅格值为 $(0, 127]$ ，此时机器人处于障碍物附近，属于危险警戒区，进入此区域，将来可能会发生碰撞；

自由区域：栅格值为 0，此处无障碍，机器人可以自由通行；

未知区域：栅格值为 255，还没探明是否有障碍物。

综上所述，代价地图通过多层融合与代价值量化，为 ROS 导航系统提供了环境感知的核心支持。其膨胀机制与动态更新能力，是解决碰撞问题、保障机器人安全运行的关键技术。

3. move_base

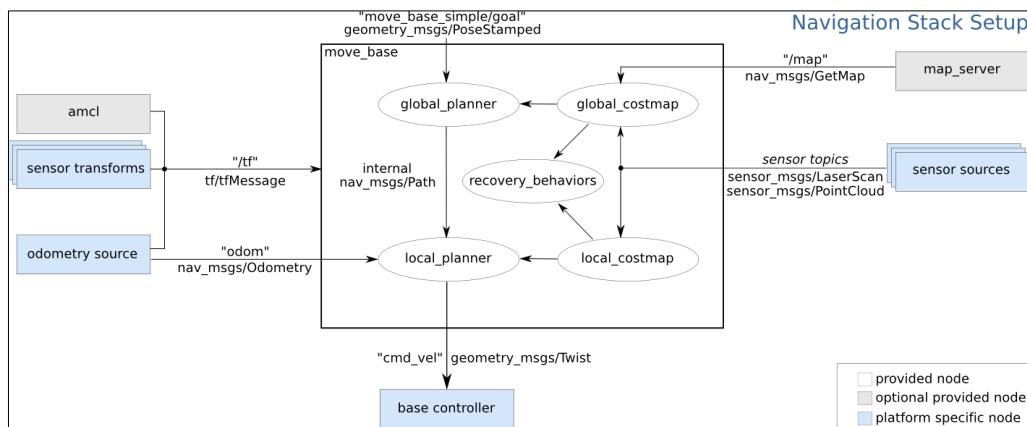
move_base 是 ROS 导航栈的核心节点与“大脑”，负责宏观调控，将定位、地图、全局规划器和局部规划器整合在一起。节点监听到目标点后，调用全局规划器生成宏观路径，再指挥局部规划器生成实时的速度指令（cmd_vel）驱动机器人，同时在机器人迷路或卡住时触发恢复行为。

功能包的核心节点是 move_base，相关节点订阅发布的话题以及相关参数可查阅网址：https://wiki.ros.org/move_base。

move_base 功能包的使用高度依赖于的一组 YAML 参数配置文件，并加载到与 move_base 节点相关的启动文件中，具体配置指南和参数详解可参考网址：<https://wiki.ros.org/navigation/Tutorials/RobotSetup>。

4. ROS 导航栈

下图是 ROS 官方提供的导航功能包集框架，清晰地描绘了移动机器人从感知、定位到路径规划与控制，从而实现自主导航的完整数据流与模块协作关系。



ROS 导航栈的架构主要划分为三类节点：核心控制节点（白色）属于 move_base 内部模块，是系统运行的必选项；平台相关节点（浅蓝色）需要根据实际硬件进行适配；可选工具节点（灰色）则是 ROS 提供的标准辅助工具。

平台相关节点是机器人的感知与执行层。左侧的“odometry source”提供里程计数据，用于估算机器人的相对运动；“sensor transforms”负责处理坐标变换，确保所有传感器数据在统一坐标系下；“sensor sources”通过传感器（如激光雷达）实时采集环境数据，用于障碍物检测；“base controller”接收 move_base 输出的速度指令，驱动机器人底盘运动。

可选工具节点是导航的定位与地图层。“map_server”加载静态地图，为导航提供环境参考；“amcl”通过粒子滤波算法，结合已知地图与传感器数据，实现机器人在环境中的自适应定位。

位于中央的“move_base”节点是导航框架的核心控制节点，它集成了全局与局部路径规划器，并实时维护着全局和局部两张代价地图。其工作流程为：用户发送目标点后，move_base 结合传感器数据与静态地图，实时更新代价地图，在获取机器人当前位姿的基础上，分别进行全局路径规划与局部轨迹规划，最终输出 cmd_vel 速度指令给底盘控制器，从而实现自主导航。代价地图的实时更新机制确保了机器人能动态避开障碍物；同时，当规划器陷入困境时，内置的“recovery_behaviors”模块会自动执行如原地旋转、清除代价地图等恢复策略，极大地提升了系统的鲁棒性。

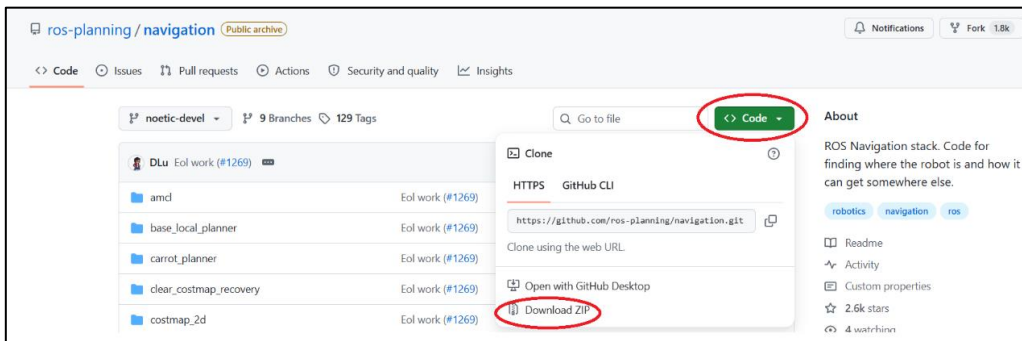
四、实验内容

内容描述：添加 move_base 功能包，实现仿真小车的自主导航与避障。

具体流程如下：

第 1 步：准备工作

1) 从 GitHub 官方仓库上下载 ROS 的导航功能包集 navigation 到本地计算机。下载网址为：<https://github.com/ros-planning/navigation>。



也可在终端上运行以下命令：

```
$ git clone https://github.com/ros-planning/navigation.git
```

2) 安装依赖 tf2-sensor-msgs 和 move-base-msgs

在终端上运行以下命令：

```
$ sudo apt install ros-noetic-tf2-sensor-msgs  
$ sudo apt install ros-noetic-move-base-msgs
```

上述两个功能包是 ROS 导航栈能够正常编译和运行的基础依赖。tf2-sensor-msgs 是解决传感器数据“坐标换算”问题的工具库，用于在“坐标变换系统（TF2）”和“传感器消息（Sensor Msgs）”之间进行数据转换；move-base-msgs 是定义“导航指令格式”的说明书，包含了 move_base 节点运行所需的消息（Topic）定义和动作（Action）定义。

3) 新建目录 param

在工作空间 catkin_ebot_sim 的源文件 src 目录下新建文件夹 param，用于存放实现机器人路径规划的配置文件。命令如下：

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup  
$ mkdir param && cd param
```

第 2 步：添加 move_base 功能包

将 navigation 文件夹里的 move_base、costmap_2d、nav_core、base_local_planner、navfn、clear_costmap_recovery、rotate_recovery、voxel_grid 这 8 个功能包直接复制到工作空间的 src/ebot_navigation 路径下。

在终端上运行以下命令（以 move_base 功能包为例）：

```
$ cp -r /path/to/navigation/move_base ~/catkin_ebot_sim/src/ebot_navigation
```

第 3 步：编写与代价地图相关的配置文件

1) 配置通用参数

编写配置文件 `costmap_common_params.yaml`，该文件是 `move_base` 在进行全局路径规划与局部路径规划时调用的通用参数，包括：机器人的尺寸、距离障碍物的安全距离、传感器信息等。

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup/param
$ gedit costmap_common_params.yaml
```

配置参考如下：

```
robot_radius: 0.12 #圆形
# footprint: [[-0.12, -0.12], [-0.12, 0.12], [0.12, 0.12], [0.12, -0.12]] #其他形状

obstacle_range: 3.0
raytrace_range: 3.5

inflation_radius: 0.2    #膨胀半径
cost_scaling_factor: 3.0 #代价缩放因子

#地图类型
map_type: costmap
#导航包所需要的传感器
observation_sources: scan

scan: {sensor_frame: laser, data_type: LaserScan, topic: scan, marking: true, clearing: true}
```

2) 配置全局代价地图参数

编写配置文件 `global_costmap_params.yaml`，该文件用于设置全局代价地图参数。

```
$ gedit global_costmap_params.yaml
```

配置参考如下：

```
global_costmap:
  global_frame: map #地图坐标系
  robot_base_frame: base_footprint #机器人坐标系
  # 以此实现坐标变换

  update_frequency: 1.0 #代价地图更新频率
  publish_frequency: 1.0 #代价地图的发布频率
  transform_tolerance: 0.5 #等待坐标变换发布信息的超时时间

  static_map: true # 是否使用一个地图或者地图服务器来初始化全局代价地图，如果不使用静态地图，这个参数为 false.
```

3) 配置局部代价地图参数

编写配置文件 `local_costmap_params.yaml`，该文件用于设置局部代价地图参数。

```
$ gedit local_costmap_params.yaml
```

配置参考如下：

```
local_costmap:
  global_frame: odom #里程计坐标系
  robot_base_frame: base_footprint #机器人坐标系

  update_frequency: 10.0 #代价地图更新频率
  publish_frequency: 10.0 #代价地图的发布频率
  transform_tolerance: 0.5 #等待坐标变换发布信息的超时时间
```

```
static_map: false #不需要静态地图, 可以提升导航效果
rolling_window: true #是否使用动态窗口, 默认为 false, 在静态的全局地图中, 地图不会变化
width: 3 # 局部地图宽度 单位是 m
height: 3 # 局部地图高度 单位是 m
resolution: 0.05 # 局部地图分辨率 单位是 m, 一般与静态地图分辨率保持一致
```

第 4 步: 编写与局部规划器相关的配置文件

编写配置文件 `base_local_planner_params.yaml`, 该文件用于基本的局部规划器参数配置, 设定了机器人的最大和最小速度限制值, 也设定了加速度的阈值。

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup/param
$ gedit base_local_planner_params.yaml
```

配置参考如下:

```
TrajectoryPlannerROS:

# Robot Configuration Parameters
max_vel_x: 0.5 # X 方向最大速度
min_vel_x: 0.1 # X 方向最小速速

max_vel_theta: 1.0 #
min_vel_theta: -1.0
min_in_place_vel_theta: 1.0

acc_lim_x: 1.0 # X 加速限制
acc_lim_y: 0.0 # Y 加速限制
acc_lim_theta: 0.6 # 角速度加速限制

# Goal Tolerance Parameters, 目标公差
xy_goal_tolerance: 0.10
yaw_goal_tolerance: 0.05

# Differential-drive robot configuration
# 是否是全向移动机器人
holonomic_robot: false

# Forward Simulation Parameters, 前进模拟参数
sim_time: 0.8
vx_samples: 18
vtheta_samples: 20
sim_granularity: 0.05
```

第 5 步: 编写 launch 文件

1) 编写与 move_base 节点相关的 launch 文件

在终端上运行以下命令:

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup/launch
$ gedit nav02_path.launch
```

nav02_path.launch 文件输入内容示例如下:

```
<launch>

  <node      pkg="move_base"      type="move_base"      respawn="false"
```



```

name="move_base" output="screen" clear_params="true">
  <rosparam file="$(find ebot_bringup)/param/costmap_common_params.
yaml" command="load" ns="global_costmap" />
  <rosparam file="$(find ebot_bringup)/param/costmap_common_params.
yaml" command="load" ns="local_costmap" />
  <rosparam file="$(find ebot_bringup)/param/local_costmap_params.yaml"
command="load" />
  <rosparam file="$(find ebot_bringup)/param/global_costmap_params.yaml"
command="load" />
  <rosparam file="$(find ebot_bringup)/param/base_local_planner_params.
yaml" command="load" />
</node>
</launch>

```

2) 编写测试文件

编写用于测试导航功能的 launch 文件 test_nav.launch，在终端上运行以下命令：

```

$ cd ~/catkin_ebot_sim/src/ebot_bringup/launch
$ gedit test_nav.launch

```

test_nav.launch 文件输入内容示例如下：

```

<launch>
  <!-- 设置地图的配置文件 -->
  <arg name="map" default="nav.yaml" />

  <!-- 运行地图服务器，并且加载设置的地图-->
  <node name="map_server" pkg="map_server" type="map_server"
args="$(find ebot_bringup)/maps/$(arg map)"/>

  <!-- 加载 amcl 文件 -->
  <include file="$(find ebot_bringup)/launch/nav01_amcl.launch" />

  <!-- 加载 path 文件，启动 move_base 路径规划与控制节点 -->
  <include file="$(find ebot_bringup)/launch/nav02_path.launch" />

</launch>

```

第 6 步：编译执行

编译并加载环境变量，在终端上运行以下命令：

```

$ cd ~/catkin_ebot_sim
$ catkin_make
$ source ./devel/setup.bash

```

加载机器人模型、雷达和 gazebo 仿真环境，并启动 rviz，终端 1 上运行以下命令：

```

$ roslaunch ebot_bringup bringup.launch

```

运行导航测试文件，启动地图服务、amcl 定位节点以及 move_base 节点，终端 2 上

运行以下命令：

```
$ roslaunch ebot_bringup test_nav.launch
```

在启动的 rviz 中，①添加 **RobotModel**、**TF**、**LaserScan** 显示，可视化机器人模型与坐标变换关系；②添加 **Map** 显示，设置 topic 为 /map，可视化静态栅格地图；③二次添加 **Map** 显示，修改显示名（Display Name）为 global_costmap，设置 topic 为 /move_base/global_costmap/costmap，可视化全局代价地图；④三次添加 **Map** 显示，修改显示名为 local_costmap，设置 topic 为 /move_base/local_costmap/costmap，可视化局部代价地图；⑤添加 **Path** 显示，修改显示名为 global_path，设置 topic 为 /move_base/NavfnROS/plan，可视化全局规划路径；⑥二次添加 **Path** 显示，修改显示名为 local_path，设置 topic 为 /move_base/TrajectoryPlannerROS/local_plan，可视化局部规划路径。

第 7 步：导航实现

添加完显示后，通过 RViz 工具栏的 2D Nav Goal 设置目的地实现导航。

五、实验要求

1、基础实验

在移动机器人平台的工作空间 catkin_ebot 下，添加 move_base 功能包，修改配置文件参数，实现移动机器人的自主导航与避障：

1) 加载静态地图与定位：在 ROS 环境中启动 map_server 加载实验 3 中构建的静态环境地图，并运行 AMCL 定位节点，准确估计并发布机器人的初始位姿。

注：如果定位的初始位姿不准确，可以在 RViz 中通过“2D Pose Estimate”手动指定并校准机器人的初始位姿。

2) 配置与启动导航栈：编写 move_base 功能包所需的各类配置文件（包括全局/局部代价地图参数、局部规划器参数等），启动 move_base 节点并加载配置文件，完成整体自主导航框架的启动。

3) 实现自主导航与避障：在 RViz 可视化界面中，通过“2D Nav Goal”发布目标点坐标，移动机器人基于规划出的全局与局部路径，自主行驶至目标点，并能够实时动态标记并避开路径上新增的障碍物。

2、进阶实验

1) 代价地图参数调优与对比

针对 costmap_common_params.yaml 中的 inflation_radius 和 cost_scaling_factor 参数，分别设置“保守”（大半径）和“激进”（小半径）两组参数，观察并截图机器人在狭窄通道中的路径规划差异，分析膨胀层对避障安全性的影响。

2) 规划器性能测试与对比

尝试切换全局或局部规划器算法，观察并记录、对比机器人自主导航过程中的

行驶轨迹、平稳性和总耗时等。

3) 动态避障功能测试

在机器人导航过程中,人为在路径上放置或移动障碍物,验证局部代价地图是否能实时更新,并观察机器人是否能触发局部重规划,平滑地绕过动态障碍物继续前往终点。

4) 异常处理与恢复行为验证

故意将机器人放置在死胡同,或设置一个被障碍物完全包围的不可达目标点,观察 `move_base` 的状态机变化,验证其是否能按顺序触发恢复策略,并在终端日志中记录触发的恢复行为名称。

3、思考问题

1) 通过 RViz 工具栏中“2D Nav Goal”按钮发布的目标点坐标,对应导航框架中哪个话题?目标点坐标还有其他发布方式吗?如果有,请列出至少 1 种。

2) 全局路径规划与局部路径规划的区别有哪些?

3) 为什么导航系统需要同时维护“全局代价地图”和“局部代价地图”?两张地图的作用分别是什么?

4) 请给出 Action (动作) 通信与 Topic (话题)、Service (服务) 通信的区别。

六、实验报告要求

1. 提交时间:两周之内提交

2. 页数:4~6 页以内

3. 报告内容:实验目的、实验步骤与结果分析、结论与问题讨论

4. 提交方式:电子版和纸质版均需提交;

电子版:实验报告和代码打包,加密压缩后,上传至网盘 <http://58.206.101.71:8266/>;

压缩包名:班级+学号+姓名,报告格式为 word 或 pdf;

纸质版:附上电子版密码,黑白双面打印即可,由各班学委收齐后交至科学馆 303。

七、参考资料

1. 张新钰,赵虚左,丘楠,郭世纯,《ROS 机器人理论与实践》,清华大学出版社

2. 课程资料:

<http://www.autolabor.com.cn/book/ROSTutorials/>

3. 课程讲解 (B 站):

https://www.bilibili.com/video/BV1Ci4y1L7ZZ/?spm_id_from=333.999.0.0&vd_source=4913ab2f45996351f46d15faddba2bfd

注:本次实验主要涉及《ROS 机器人理论与实践》第七章和第十章内容,重点参阅 7.2.4、10.1 内容。